Séance 4 · Git, HTTP, REST et FastAPI

MaintAI reste l'exemple de toute la séance. Vous rendez d'abord votre TP0 MaintAI sur GitHub, puis le modèle MaintAI devient une *API* : un autre programme lui envoie un relevé de capteurs par le réseau et reçoit la probabilité de panne. On commence par finir l'exercice CantineAI resté incomplet en séance 3.

Étape	Ce que vous faites	Durée
1. CantineAI	finir l'exercice incomplet de la S3 (réglages, tableau)	25 min
2. TP0 MaintAI	le rapport MaintAI de chacun, sur GitHub, dans `TP/TP0`	45 min
3. HTTP et REST	comment un programme parle à un autre	25 min
4. FastAPI	votre première API, en 15 lignes	25 min
5. TP1	l'API de prédiction MaintAI, jusqu'à 13 tests verts	55 min
6. Ticket	quatre questions, sur papier	5 min

À rendre pour cette séance : sur GitHub, un rapport TP0 MaintAI par membre dans `TP/TP0` (activité 2) ; le TP1 dans `TP/TP1`, avec son `COMPTE_RENDU.md` rempli, par une pull request et le tag `tp1` (fin en début de séance 5).

Activité 1 · Finir l'exercice CantineAI de la S3

En groupe, 25 minutes. Le code de la S3 est générique : il lit ses réglages dans `config.py`. Pour passer de MaintAI à CantineAI (prédire le nombre de repas servis), **seuls les réglages changent**, pas le code.

1. Ouvrez votre projet de la S3, **environnement activé** (l'invite affiche `(.venv)`). Si votre code de la S3 n'est pas fini, prenez `projet_solution`.
2. Copiez `cantine_repas.csv` dans le dossier `data/` du projet.
3. Dans `monprojet/config.py`, changez ces six lignes, et rien d'autre :

```
DATA_PATH = ROOT / "data" / "cantine_repas.csv"
TARGET = "repas_servis"          # la colonne à prédire
TASK = "regression"           # on prédit un nombre
MAIN_METRIC = "mae"           # erreur moyenne, en repas
SPLIT = "chronologique"       # les jours les plus récents en test
TIME_COLUMN = "date"          # la colonne qui donne l'ordre
```

4. Dans le terminal, depuis la racine du projet :

```
python -m pytest              # attendu : 8 passed
python -m monprojet.train     # le tableau de comparaison
```

Recopiez le tableau obtenu :

Modèle	MAE (repas)
baseline (moyenne)	
régression linéaire	
forêt aléatoire	
gradient boosting	

La grille CantineAI. Quel est le meilleur modèle, et de combien bat-il la baseline ? Pourquoi un découpage chronologique, et pas aléatoire, pour des repas servis jour après jour ?

À montrer : le tableau de CantineAI, avec la MAE du meilleur modèle et celle de la baseline.

Si ça ne marche pas. Une erreur sur la date : `TIME_COLUMN` est oublié. 8 passed mais une MAE bien plus basse que vos voisins : `SPLIT` est resté `"random"`, le test contient des jours mélangés avec l'entraînement, et le résultat est trop optimiste. Si le code casse sur CantineAI, il dépendait de MaintAI quelque part (un nom de colonne écrit en dur) : corrigez-le.

Votre rapport TP0 MaintAI

Chaque membre écrit **son** rapport sur le TP0 MaintAI de la S3, avec ses propres phrases, dans un fichier Markdown (`.md`). GitHub affiche un fichier `.md` mis en forme. Modèle à suivre (les chiffres sont ceux de la S3) :

```
# TP0 : MaintAI
**Nom :** Prénom Nom · **Groupe :** G3

## Le problème
Prédire une panne à partir de 4 capteurs ; une panne manquée arrête une ligne.

## Résultats sur le jeu de test
| Modèle | Accuracy | Rappel | Précision |
|-|-|-|-|
| baseline | 0.824 | 0.000 | 0.000 |
| régression logistique | 0.889 | 0.574 | 0.737 |

## Mon choix, et pourquoi
## Le critère (rappel >= 0,75) est-il atteint ?
## Ce qui reste flou
```

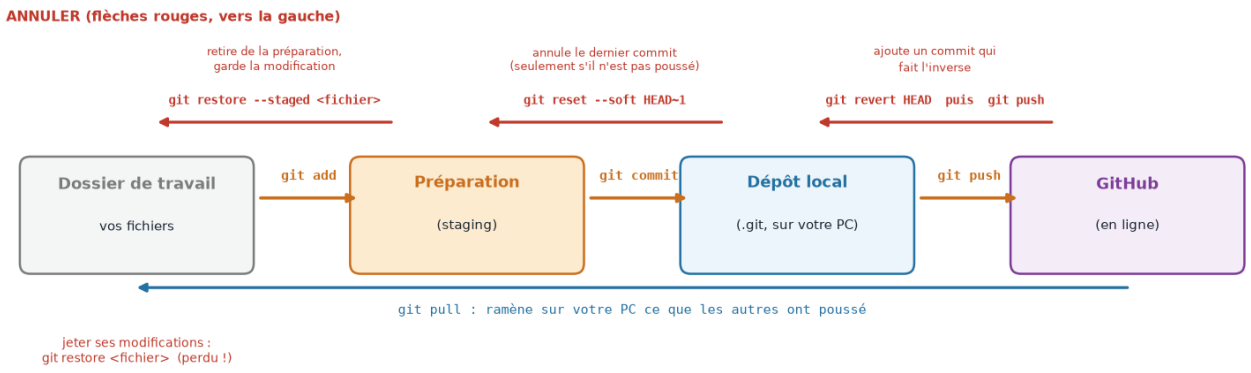
Le critère n'est pas atteint (rappel 0,574) : c'est un résultat valable, à écrire et à expliquer.

Markdown en une minute

Vous écrivez	Vous obtenez	Pour
`# Titre` ou `## Section`	un titre, un sous-titre	le titre du rapport, ses parties
`**texte**`	du texte en **gras**	le nom, le groupe
`- un élément`	une liste à puces	les points du choix
`| Modèle | Rappel |` puis `|-|-|`	un tableau	vos résultats MaintAI

L'essentiel de Git

Quatre endroits, trois commandes. `git add` prépare un fichier, `git commit` photographie le projet sur votre PC, `git push` envoie les commits sur GitHub. `git pull` ramène sur votre PC ce que les autres ont poussé. **Un commit reste sur votre machine tant qu'il n'est pas poussé.**



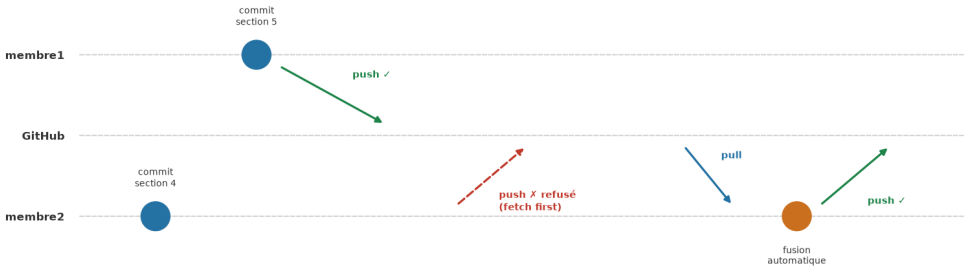
Analogie : préparer = choisir les composants à souder ; commit = photographier la carte (une version) ; push = déposer la photo dans l'armoire commune ; pull = récupérer les photos des collègues.

Annuler (les flèches rouges) :

Vous avez fait	Pour annuler	Ce qui se passe
modifié un fichier	`git restore <fichier>`	le fichier revient au dernier commit : **votre modification est perdue**
`git add <fichier>`	`git restore --staged <fichier>`	le fichier sort de la préparation ; votre modification reste dans le fichier
`git commit` (pas encore poussé)	`git reset --soft HEAD~1`	le dernier commit disparaît ; ses changements restent préparés, prêts à être recommités
`git push` (déjà sur GitHub)	`git revert HEAD`, puis `git push`	un nouveau commit qui fait l'inverse ; l'historique reste juste pour les autres

N'utilisez jamais `reset` sur un commit déjà poussé : les autres l'ont déjà. Une fois poussé, toujours `revert`. Un fichier poussé par erreur (un CSV) : `git rm --cached <fichier>`, ajoutez la règle au `.gitignore`, puis `commit` et push.

Travailler à plusieurs : `git pull` avant `git push`. Si un autre membre a poussé pendant que vous travailliez, GitHub refuse votre push (`rejected (fetch first)`) pour ne pas écraser son travail. `git pull` fusionne ses commits avec les vôtres, puis `git push` passe.



Membre2 travaillait sur une version ancienne : GitHub refuse pour ne pas écraser le travail de membre1.
Règle : git pull AVANT git push.

N'entre jamais dans Git	Pourquoi	Où il va
`.venv/`	recréé par `pip install`	nulle part
`data/*.csv`	volumineux, évolue	DVC (séance 7)
`models/`	produit par l'entraînement	MLflow (séance 7)
`.env`	contient des secrets	sur chaque machine seulement

Une règle du `.gitignore` qui contient un `/` (comme `data/*.csv`) ne vaut qu'à partir du dossier de **son** fichier `.gitignore`. Le starter du TP1 a son propre `.gitignore`, déjà rempli.

Activité 2 · Rendre le TP0 MaintAI sur GitHub

En groupe, 35 minutes.

1. **Un membre** crée le dépôt sur GitHub (« Add a README file » et « .gitignore : Python » cochés), puis invite les autres (Settings, Collaborators). Les autres acceptent l'invitation.
2. **Chacun** : `git clone <url>`, puis `cd <dépôt>`.
3. **Le même membre** crée `TP\TP0\README.md`, puis add, `commit`, push. Pourquoi un README ? **Git ne garde pas un dossier vide** : `mkdir TP\TP0` seul n'apparaît pas dans `git status`.
4. **Chacun** : `git pull`, écrit son rapport MaintAI `TP\TP0\Prenom_Nom.md`, puis add, `commit`, `pull`, push.

Les commandes, membre par membre (les lignes après # sont des explications : ne les tapez pas) :

```
# -- un membre, une seule fois --
git clone https://github.com/<compte>/<depot>.git
cd <depot>
mkdir TP\TP0
notepad TP\TP0\README.md          # écrire : # TP0 : MaintAI
git add TP
git commit -m "Crée le dossier TP/TP0"
git push

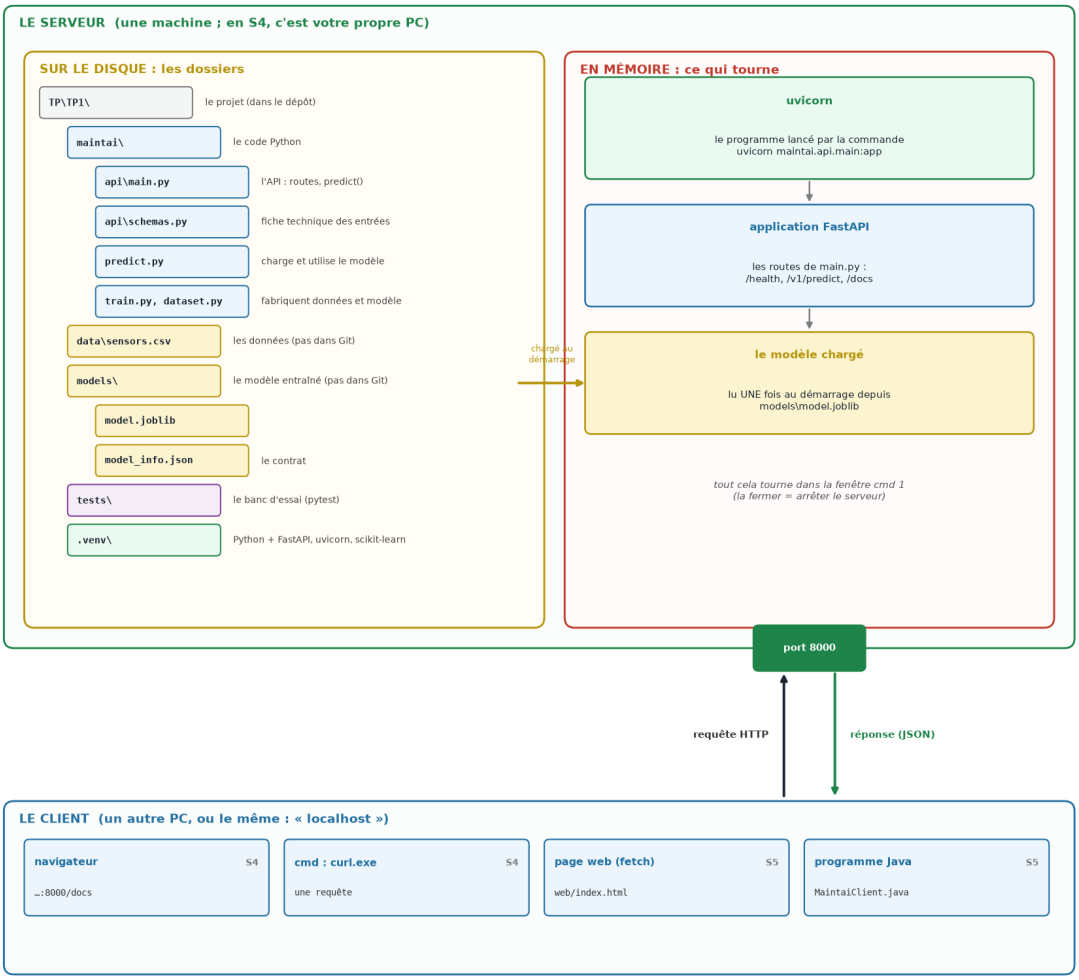
# -- chaque membre (y compris le premier) --
git pull                        # récupérer TP/TP0
notepad TP\TP0\Prenom_Nom.md       # écrire le rapport, enregistrer
git add TP\TP0\Prenom_Nom.md
git commit -m "TP0 : rapport de Prénom Nom"
git pull                        # au cas où un autre a poussé avant
git push
```

À montrer : sur GitHub, le dossier `TP/TP0` avec un rapport MaintAI par membre, et un commit de chacun.

Votre `git push` a-t-il été refusé avec `rejected (fetch first)` ? Que s'est-il passé, et qu'avez-vous fait ?

HTTP : comment un programme parle à un autre

Une API est un programme qui tourne sur un *serveur* et attend des requêtes. Un *client* (le navigateur, `curl.exe`, plus tard une page web ou un programme Java) lui envoie une requête et reçoit une réponse. En séance 4, le serveur et le client sont votre propre PC.



Analogie : le disque, c'est le magasin de pièces et de plans ; la mémoire, l'appareil assemblé et sous tension ; le port 8000, son connecteur ; les clients, les sondes branchées dessus.

Une requête HTTP contient une *méthode* (`GET` lit, `POST` envoie des données), une *URL*, des *en-têtes* (`Content-Type: application/json`, plus tard `X-API-Key`) et un *corps* en JSON. **La réponse** contient un *code de statut*, des en-têtes et un corps en JSON.

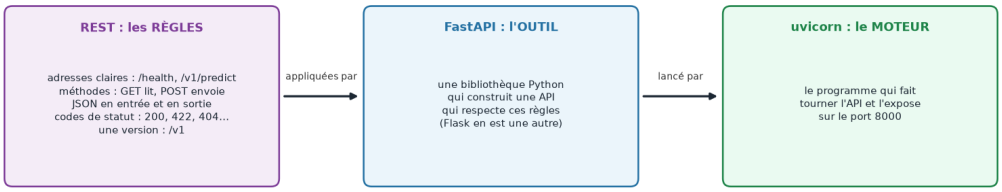
Lire une URL. `http://localhost:8000/v1/predict`

Morceau	Sens
`localhost`	la machine elle-même : le serveur tourne sur votre PC
`8000`	le *port* : la « porte » où uvicorn écoute ; un autre programme peut en utiliser une autre
`/v1/predict`	la *route* : un chemin choisi dans le code Python, **pas un dossier** sur le disque. `/v1` est la version de l'API

Code	Sens	Dans MaintAI
200	succès	prédiction renvoyée
401	non authentifié	clé API absente (séance 5)
404	route inconnue	faute de frappe dans l'URL
422	entrée invalide	température de 500 °C
500	erreur du serveur	bug dans le code
503	service indisponible	modèle non chargé

REST et FastAPI

REST est un ensemble de règles pour concevoir une API web : des URL claires (`/health`, `/v1/predict`), les méthodes HTTP, des codes précis, du JSON en entrée et en sortie, une version (`/v1` : le contrat peut évoluer sans casser les clients). **FastAPI** est la bibliothèque Python avec laquelle on écrit une API qui suit ces règles. **uvicorn** est le programme qui fait tourner cette API et l'expose sur le port 8000. On écrit donc une API REST *avec* FastAPI, lancée *par* uvicorn.



Analogie : REST est la norme (la spécification d'un bus de terrain), FastAPI le circuit construit selon la norme, uvicorn l'alimentation qui le met sous tension.

FastAPI : un schéma, trois usages

Le *schéma* est une classe Pydantic qui décrit les données attendues, comme la fiche technique d'un composant : le nom de chaque entrée, son type, sa plage admise.

```
class Reading(BaseModel):
    temperature_c: float = Field(ge=-40, le=150, description="Température (°C)")
```

Morceau	Sens
`temperature_c`	le nom du champ, exactement celui attendu dans le JSON
`: float`	le type : un nombre à virgule (`"chaud"` sera refusé)
`ge=-40`	*greater or equal* : au moins -40
`le=150`	*less or equal* : au plus 150
`description=...`	le texte affiché dans la documentation

On écrit ce schéma **une seule fois**, et FastAPI s'en sert pour trois choses :

1. **Valider.** Avant même d'appeler votre fonction, FastAPI vérifie chaque requête. Une température de 500 est refusée avec le code **422**, et la réponse dit où et pourquoi :

   ```
   {"detail": [{"loc": ["body", "temperature_c"],
       "msg": "Input should be less than or equal to 150", "input": 500}]}
   ```

   Votre fonction ne voit jamais une valeur impossible : le modèle n'est pas appelé.
2. **Documenter.** FastAPI construit seul la page `http://localhost:8000/docs` : chaque route, chaque champ avec son type, ses bornes et sa description, un exemple, et un bouton « Try it out » pour tester sans écrire de code.
3. **Donner un objet Python typé.** Dans la fonction, `reading` n'est pas un texte JSON à découper : c'est un objet. `reading.temperature_c` est un `float`, déjà vérifié entre -40 et 150. `reading.model_dump()` le transforme en dictionnaire, la forme qu'attend le modèle.

Sans schéma, il faudrait écrire à la main les `if` de contrôle, la page de documentation et la conversion du JSON, et les garder d'accord entre eux. Avec le schéma, les trois viennent du même endroit : ils ne peuvent pas se contredire.

Lancer l'API. `uvicorn demo_api:app --reload`

Morceau	Sens
`uvicorn`	le programme qui fait tourner l'API (installé dans le `.venv`)
`demo_api`	le fichier `demo_api.py`, sans `.py` ; pour MaintAI : `maintai.api.main`, c'est-à-dire `maintai/api/main.py` écrit avec des points
`:app`	la variable de ce fichier qui contient l'application : `app = FastAPI()`
`--reload`	relance l'API à chaque fois que vous enregistrez le fichier

La fenêtre où tourne uvicorn est occupée : ouvrez une **deuxième** fenêtre pour `curl.exe`. Ctrl+C arrête le serveur.

Activité 3 · Le web vu de près

En binôme, 10 minutes. Ouvrez un site web, puis les outils de développement (F12), onglet *Network* (Réseau). Rechargez la page.

Une requête : méthode _____ URL _____ code _____

Deux en-têtes : _____

Une requête qui n'est pas en 200 : son code _____, ce qu'il veut dire : _____

Quiz. Quel code renvoie l'API MaintAI pour :

Situation	Code
(a) une température de 500 °C	
(b) l'URL `/v1/predcit`	
(c) le modèle n'est pas chargé	
(d) la clé API est absente	
(e) un bug dans le code de l'API	

Activité 4 · Votre première API

En binôme, 15 minutes. `demo_api.py` est un petit fichier **à part**, pour comprendre le principe avant MaintAI : ce n'est ni `main.py` ni `schemas.py`.

1. Dans un dossier vide, avec un environnement activé : `pip install fastapi "uvicorn[standard]"`.
2. Créez `demo_api.py` (attention : Notepad peut l'enregistrer en `demo_api.py.txt`) :

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()                        # l'application

class Reading(BaseModel):              # le schéma
    temperature_c: float = Field(ge=-40, le=150)

@app.get("/health")                    # la route GET /health
def health():
    return {"status": "ok"}

@app.post("/v1/predict")               # la route POST /v1/predict
def predict(reading: Reading):         # reading : déjà validé
    return {"alert": reading.temperature_c > 80}
```

3. Lancez `uvicorn demo_api:app --reload`, puis ouvrez `http://localhost:8000/docs`.
4. « Try it out » : une température de 70, puis de 500. Codes obtenus : 70 → _____ 500 → _____
5. Dans une deuxième fenêtre cmd, la même chose sans navigateur. Écrivez `{"temperature_c": 70}` dans un fichier `releve.json`, puis :

```
curl.exe http://localhost:8000/health
curl.exe -X POST http://localhost:8000/v1/predict -H "Content-Type: application/json" -d "@releve.json"
```

   `-X POST` : la méthode ; `-H` : un en-tête ; `-d "@releve.json"` : le corps, lu dans le fichier. Sous Windows, ne mettez pas le JSON entre apostrophes dans la commande : il est mal transmis (erreur 422).
6. Ajoutez un champ `vibration_mm_s` avec ses bornes, enregistrez, rechargez `/docs`.

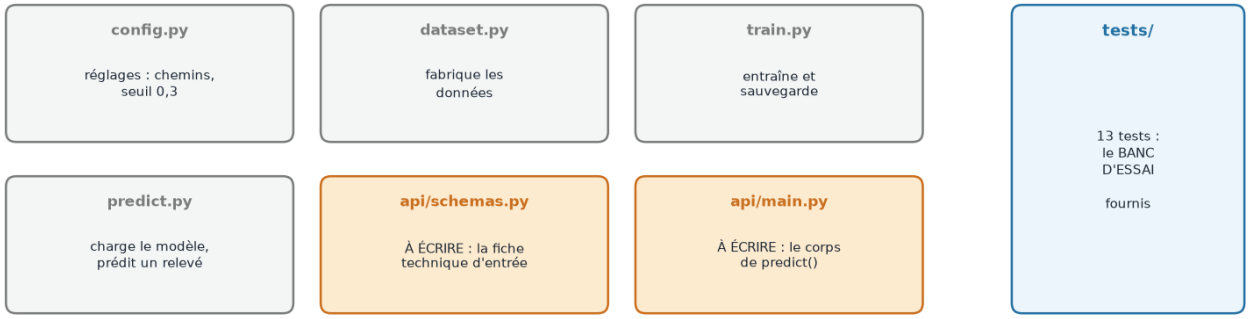
À montrer : `/docs` avec le 422 de la température de 500.

TP1 · Une API de prédiction pour MaintAI

Séance 4, fin en début de séance 5 · thèmes 2.2 et 2.3 du syllabus

Contexte. Le modèle de prédiction de panne de MaintAI doit être utilisable par d'autres programmes : un tableau de bord, une application mobile, le système de l'atelier. Il devient une API : on lui envoie un relevé de capteurs en JSON, elle renvoie la probabilité de panne, et une alerte si la probabilité atteint le seuil (0,3). Une valeur impossible doit être refusée avant d'atteindre le modèle.

Le contrat (`models/model_info.json`, écrit par l'entraînement) : les variables `temperature_c`, `vibration_mm_s`, `load_pct`, `runtime_hours` ; la cible `failure` ; le seuil `0.3`.



Gris : fourni, à lire seulement. Orange : les deux trous à combler. Bleu : le banc d'essai qui dit si c'est bon.

Mise en place dans le dépôt : TP/TP1 (10 min)

Le TP1 se range à côté du TP0, dans `TP\TP1`, sur une branche `tp1`.

```
# -- un membre --
git switch main
git pull
git switch -c tp1                    # une branche pour le TP1
# copier le contenu de S4_maintai-starter-tp1.zip dans TP\TP1
git add TP
git commit -m "TP1 : starter"
git push -u origin tp1

# -- chaque membre --
git pull
git switch tp1
cd TP\TP1
python -m venv .venv
.venv\Scripts\activate.bat          # PowerShell : .venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m maintai.dataset            # écrit data\sensors.csv
python -m maintai.train              # écrit models\model.joblib
python -m pytest                     # attendu : 3 failed, 10 passed
```

Un environnement par dossier : celui de la S3 n'a pas FastAPI. Le `.gitignore` de `TP\TP1` exclut déjà `.venv`, `data/*.csv` et `models/`.

Attention : au départ, deux tests qui attendent un 422 passent déjà sans rien prouver. Le schéma est vide et refuse tout champ inconnu (`extra="forbid"`) : tout relevé est refusé, même un bon. Un test vert ne suffit pas, il faut savoir *pourquoi* il passe.

Sprint : l'API de prédiction (35 min)

1. **`maintai/api/schemas.py`** : remplacez `pass` par les 4 champs de `SensorReading`, de type `float`, avec des bornes réalistes. Le premier est donné :

   ```
   temperature_c: float = Field(ge=-40, le=150, description="Température du moteur (°C)")
   vibration_mm_s: float = Field(ge=..., le=..., description="...")
   load_pct: float = Field(ge=..., le=..., description="...")
   runtime_hours: float = Field(ge=..., le=..., description="...")
   ```

`pytest` donne encore 3 failed, mais l'erreur devient `NotImplementedError` : le schéma accepte maintenant les relevés valides, et la requête arrive jusqu'à `predict()`.

2. **`maintai/api/main.py`** : le corps de `predict()`. Récupérez le `predictor` dans `state` ; s'il est absent, levez `HTTPException` avec le code 503 ; sinon, renvoyez sa prédiction pour `reading.model_dump()`.
3. `uvicorn maintai.api.main:app --reload`, puis `/docs` : un relevé normal, puis 500 °C.
4. `python -m pytest` : **13 passed**. Committez sur la branche `tp1` au fil de l'eau.

À montrer : 13 passed, et le 422 de 500 °C dans `/docs`.

Pistes, sans la solution

À écrire	Piste
les bornes	des bornes **physiques** : ce qu'un moteur réel peut afficher, pas le minimum et le maximum de `sensors.csv`
le predictor	il est chargé **une fois**, au démarrage, dans le dictionnaire `state` : `state.get("predictor")` vaut `None` si le modèle manque
le 503	`raise HTTPException(status_code=503, detail="...")` ; un message qui dit quoi faire
la prédiction	`predictor.predict(...)` attend un dictionnaire : `reading.model_dump()` ; la réponse se construit avec `PredictionOut(**...)`

Les erreurs les plus fréquentes. Des bornes copiées sur les données : une machine réelle un peu plus chaude serait refusée. Un champ de type `int` : 15000,5 heures serait refusé. Un `predict()` qui ne vérifie pas le modèle : 500 au lieu de 503. `uvicorn` ou `pytest` lancé ailleurs que dans `TP\TP1`.

Décision : l'ADR 002

Copiez `docs/adr/000-modele.md` en `docs/adr/002-validation.md` et justifiez deux décisions :

— **les bornes de chaque variable**, justifiées physiquement ;
— **modèle absent** : répondre 503, ou refuser de démarrer ? Qu'est-ce que chaque option fait perdre ?

Le compte rendu : COMPTE_RENDU.md

Le starter contient un fichier `COMPTE_RENDU.md`, dans `TP\TP1`. Remplissez-le au fil du TP, pas à la fin : il demande les sorties de `pytest` à chaque étape, vos bornes et leur justification, votre `predict()`, les codes obtenus pour quatre requêtes, cinq questions de compréhension et le lien de votre pull request. Il est rendu avec le code, dans la même pull request.

Rendre le TP1 (10 min)

```
git push                       # la branche tp1
# GitHub : Compare & pull request
# un coéquipier relit, commente, approuve
# Merge pull request (Create a merge commit)
git switch main
git pull
git tag tp1
git push --tags
```

Le relecteur vérifie : `pytest` passe chez lui ; ni données, ni modèle, ni `.venv` dans les changements ; l'ADR 002 justifie les bornes ; `COMPTE_RENDU.md` est rempli ; au moins un commentaire.

Critères de réussite. 13 tests passent ; `/docs` montre l'exemple et la description de chaque champ ; 500 °C renvoie 422 ; modèle absent renvoie 503 ; l'ADR 002 et le compte rendu sont dans `TP/TP1` ; la pull request est fusionnée et le tag `tp1` existe.

En cas de problème

Symptôme	Solution
`git push` : `403`, permission refusée	acceptez l'invitation au dépôt (notifications GitHub)
une fenêtre du navigateur s'ouvre au premier push	normal : connectez-vous à GitHub
`rejected (fetch first)`	quelqu'un a poussé avant vous : `git pull`, puis `git push`
un éditeur étrange s'ouvre au `git pull`	c'est le message de fusion : tapez `:wq` puis Entrée
`TP\TP0` n'apparaît pas dans `git status`	Git ignore les dossiers vides : créez un fichier dedans
`.venv/` ou un CSV dans `git status`	complétez le `.gitignore` ; déjà ajoutés : `git restore --staged <chemin>` ; déjà poussés : `git rm --cached <chemin>`, commit, push
`&&` refusé dans PowerShell	tapez les deux commandes sur deux lignes
`uvicorn` introuvable	activez l'environnement de ce dossier
`ModuleNotFoundError: maintai`	lancez depuis `TP\TP1`
port 8000 déjà utilisé	un autre uvicorn tourne : Ctrl+C, ou `--port 8001`
`/health` : `"model_loaded": false`	`python -m maintai.train`, puis relancez uvicorn
`curl` : `422 JSON decode error`	écrivez le JSON dans `releve.json` et envoyez-le avec `-d "@releve.json"`, ou testez dans `/docs`

× découpez ici

Ticket de sortie · Séance 4

Nom : _____ Groupe : _____

1. Que fait `git pull`, et quand faut-il le lancer ?

2. Quel code renvoie l'API pour une température de 500 °C, et qui écrit ce contrôle ?

3. Pourquoi rendre le TP par une pull request plutôt que pousser sur `main` ?

4. Qu'est-ce qui reste flou pour vous ?